

**VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY**



Huynh Anh Tu — 521H0325
**Object-Oriented Solutions to the Problem of Mining
Maximal Frequent Itemsets from Accumulated
Dynamic Uncertain Databases**

SENIOR PROJECT REPORT

Supervisor: Dr. Nguyen Chi Thien

HO CHI MINH CITY, 2026

**VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY**



Huynh Anh Tu — 521H0325
**Object-Oriented Solutions to the Problem of Mining
Maximal Frequent Itemsets from Accumulated
Dynamic Uncertain Databases**

SENIOR PROJECT REPORT

Supervisor: Dr. Nguyen Chi Thien

HO CHI MINH CITY, 2026

ACKNOWLEDGEMENTS

We would like to sincerely thank Dr. Nguyen Chi Thien for the guidance, support, and valuable feedback throughout the course of this project. We also thank the Faculty of Information Technology at Ton Duc Thang University for providing the academic environment and resources necessary to complete this work.

We are grateful to our families and friends for their continued encouragement and support during the research and writing process.

Ho Chi Minh City, 5 / 6 / 2026

Authors

(Sign and write full name)

Huynh Anh Tu

DECLARATION OF INTEGRITY

THIS PROJECT WAS COMPLETED AT TON DUC THANG UNIVERSITY

We hereby declare that this project was carried out by ourselves under the scientific guidance of Dr. Nguyen Chi Thien. The research content and results presented in this project are truthful and have not been published in any form previously. The data presented in tables and used for analysis, comments, and evaluation were collected by the authors from various sources, which are clearly cited in the references section.

In addition, the project uses some comments, evaluations, and data from other authors and organizations, all of which are cited with proper attribution.

If any fraud is discovered, we accept full responsibility for the content of the project. Ton Duc Thang University has no responsibility for any copyright violations caused by us during the implementation process (if any).

Ho Chi Minh City, 5 / 6 / 2026

Authors

(Sign and write full name)

Huynh Anh Tu

ABSTRACT

This project presents an object-oriented software system for mining maximal frequent itemsets from accumulated dynamic uncertain databases. Real-world data frequently contains uncertainty — items exist with an associated existential probability rather than as definite facts — and databases grow over time as new transaction batches are accumulated.

Three algorithms are designed and implemented following object-oriented principles: (1) U-Apriori, an exact breadth-first search method based on candidate generation; (2) UH-Mine, an exact depth-first search method based on a hyperlinked structure; and (3) ACO-Miner, an approximate method based on Ant Colony Optimization. All algorithms share a common abstract base class and operate on a unified uncertain database model that supports dynamic accumulation of transaction batches.

Experiments on retail transaction datasets of varying sizes demonstrate that UH-Mine consistently outperforms U-Apriori at lower support thresholds while producing identical exact results. ACO-Miner offers a configurable speed-completeness trade-off suitable for large-scale approximate mining.

Contents

ACKNOWLEDGEMENTS	i
DECLARATION OF INTEGRITY	ii
ABSTRACT	iii
LIST OF FIGURES	vi
LIST OF TABLES	vii
LIST OF ABBREVIATIONS	viii
1 INTRODUCTION AND RESEARCH OVERVIEW	1
1.1 Reasons for Choosing the Topic	1
1.2 Review of Existing Approaches and Proof of Frequency Criteria	1
1.2.1 Formal Definitions	1
1.2.2 The Anti-Monotone Property	2
1.2.3 U-Apriori — Proof of Correctness	2
1.2.4 UH-Mine — Proof of Equivalence to U-Apriori	2
1.2.5 ACO-Miner — Soundness and Approximation Bound	3
1.2.6 Algorithm Comparison Summary	3
1.3 Research Objectives	3
2 THEORETICAL BASIS	3
2.1 Problem Definition	3
2.2 System Overview	4
2.3 Data Package	5
2.3.1 Transaction Class	5
2.3.2 Database Class	6
2.4 Algorithm Package	7
2.4.1 Algorithm Abstract Base Class	7
2.4.2 AlgorithmFactory Class	8
2.4.3 U-Apriori	9
2.4.4 UH-Mine	10
2.4.5 ACO-Miner	10
2.5 Output Package	12
2.5.1 ResultWriter	12
2.5.2 ResultComparator	12
2.5.3 BenchmarkLogger	13
2.6 Accumulated Dynamic Database Model	14
2.7 Class Summary	15

3	PROPOSED MODEL	15
3.1	Hardware and Software Environment	15
3.2	Datasets	15
3.3	Evaluation Metrics	16
3.4	Dynamic Accumulation Experiment	16
4	EXPERIMENTS	17
4.1	Algorithm Correctness Verification	17
4.2	Runtime Performance	17
4.3	Effect of Minimum Support Threshold	18
4.4	Dynamic Accumulation Results	19
4.5	ACO-Miner Coverage Analysis	20
4.6	Discussion	21
5	CONCLUSION	21
5.1	Conclusion	21
5.2	Contributions	21
5.3	Limitations	22
5.4	Future Development	22

List of Figures

1	Package diagram of the proposed system	4
2	Package diagram of the proposed system	4
3	Package diagram of the proposed system	5
4	Sequence diagram of the mining process	14
5	Runtime comparison (log scale, $\theta = 0.05$)	17
6	Runtime vs. θ (88,162 transactions)	18
7	Runtime across dynamic accumulation steps ($\theta = 0.05$)	20
8	ACO-Miner coverage vs. $A \times I$ ($\theta = 0.05$)	20

List of Tables

1	U-Apriori worked example for $X = \{40, 49\}$	2
2	Theoretical properties of the three algorithms	3
3	Class responsibilities	15
4	Experimental datasets	15
5	Correctness verification ($\theta = 0.05$)	17
6	Runtime comparison ($\theta = 0.05$, ms)	17
7	Effect of θ on full dataset (88,162 transactions)	18
8	Dynamic accumulation results ($\theta = 0.05$)	19
9	Itemset changes tracked by ResultComparator ($\theta = 0.05$)	19
10	ACO coverage vs. runtime (retail_uncertain_5000.txt, $\theta = 0.05$, exact = 4 itemsets)	20

LIST OF ABBREVIATIONS

ACO	Ant Colony Optimization
BFS	Breadth-First Search
CLI	Command-Line Interface
DB	Database
DFS	Depth-First Search
ExpSup	Expected Support
OOP	Object-Oriented Programming
TDTU	Ton Duc Thang University
UH	Uncertain Hyperlinked
UF	Uncertain Frequent

CHAPTER 1. INTRODUCTION AND RESEARCH OVERVIEW

1.1 Reasons for Choosing the Topic

Data mining is the process of discovering patterns and useful knowledge from large datasets. One of the most fundamental tasks in data mining is frequent itemset mining — identifying collections of items that appear together frequently across a set of transactions. Frequent itemset mining has been applied widely in market basket analysis, medical diagnosis, intrusion detection, and recommendation systems [2].

Classical frequent itemset mining assumes that data is certain — each item either exists in a transaction or does not. However, real-world data is often inherently uncertain. Sensor readings contain measurement error, medical test results carry probabilistic diagnoses, and RFID-based inventory systems record item presence with confidence scores rather than binary flags [4]. Such data is modeled as an *uncertain database* where each item in a transaction carries an *existential probability*.

A further challenge arises when databases are not static but grow over time. In many practical applications — retail logs, web clickstream data, medical records — new batches of transactions are continually appended to the historical database. This *accumulated dynamic* model requires that mining results be updated as the database grows without discarding historical data. Mining *maximal* frequent itemsets — those with no frequent proper superset — produces the most compact and informative result representation.

Despite significant prior work on individual algorithms for uncertain data, no unified object-oriented system has addressed both the uncertain data model and the accumulated dynamic model together in a single extensible framework. This project addresses that gap.

1.2 Review of Existing Approaches and Proof of Frequency Criteria

1.2.1 Formal Definitions

Definition 1 — Uncertain Database

$D = \{t_1, t_2, \dots, t_n\}$ is a collection of n uncertain transactions. Each item x in transaction t_i has an *existential probability* $p(x, t_i) \in (0, 1]$.

Definition 2 — Expected Support

The expected support of itemset X over database D is:

$$\text{ExpSup}(X, D) = \frac{1}{n} \sum_{i=1}^n \prod_{x \in X} p(x, t_i) \quad (1)$$

where the product equals 0 if any item in X is absent from t_i .

Definition 3 — Frequent Itemset

Itemset X is *frequent* in D with threshold θ if $\text{ExpSup}(X, D) \geq \theta$.

Definition 4 — Maximal Frequent Itemset

A frequent itemset X is *maximal* if no proper superset $X' \supset X$ is also frequent.

1.2.2 The Anti-Monotone Property**Theorem 1 — Anti-Monotone Property**

For any two itemsets X and Y where $X \subsetneq Y$: $\text{ExpSup}(Y) \leq \text{ExpSup}(X)$.

Proof. Let $Y = X \cup \{y\}$. For any t_i : $P(Y \subseteq t_i) = P(X \subseteq t_i) \cdot p(y, t_i) \leq P(X \subseteq t_i)$ since $p(y, t_i) \leq 1$. Summing and dividing by n gives the result. $\square$

Corollary. If X is not frequent, no superset of X can be frequent — pruning is safe in all exact algorithms.

1.2.3 U-Apriori — Proof of Correctness

U-Apriori [5] computes expected support by direct database scan (Equation 1). A worked example for $X = \{40, 49\}$ over 4 transactions:

Table 1: U-Apriori worked example for $X = \{40, 49\}$

Transaction	Items	$P(X \subseteq t_i)$
t_1	{40:0.9, 49:0.8, 42:0.7}	$0.9 \times 0.8 = 0.72$
t_2	{40:0.6, 42:0.5}	0 (49 absent)
t_3	{49:0.7, 42:0.8}	0 (40 absent)
t_4	{40:0.8, 49:0.9}	$0.8 \times 0.9 = 0.72$
$\text{ExpSup}(\{40, 49\}) = (0.72 + 0 + 0 + 0.72)/4$		= 0.36

Since $0.36 \geq \theta = 0.05$, itemset $\{40, 49\}$ is **frequent**. U-Apriori is **sound** (direct computation, no approximation) and **complete** (Apriori pruning via Corollary never discards a frequent itemset).

1.2.4 UH-Mine — Proof of Equivalence to U-Apriori

UH-Mine [7] restricts summation to the intersection of relevant transaction lists:

$$\text{ExpSup}_{\text{UH-Mine}}(X) = \frac{1}{n} \sum_{i: t_i \supseteq X} \prod_{x \in X} p(x, t_i) = \text{ExpSup}_{\text{U-Apriori}}(X) \quad (2)$$

UH-Mine is **exact and complete** — mathematically identical to U-Apriori.

1.2.5 ACO-Miner — Soundness and Approximation Bound

ACO-Miner [8] selects items with probability:

$$P(\text{select } i) = \frac{\tau_i^\alpha \cdot \eta_i^\beta}{\sum_j \tau_j^\alpha \cdot \eta_j^\beta} \quad (3)$$

where τ_i is pheromone and $\eta_i = \text{ExpSup}(\{i\})$. ACO-Miner only adds item y when $\text{ExpSup}(\text{prefix} \cup \{y\}) \geq \theta$ — every reported itemset is genuinely frequent (**sound**). The probability of missing itemset X is:

$$P(X \text{ never found}) = (1 - P(\text{ant discovers } X))^{A \cdot I} \quad (4)$$

which approaches 0 as $A \cdot I$ grows.

1.2.6 Algorithm Comparison Summary

Table 2: Theoretical properties of the three algorithms

Property	U-Apriori	UH-Mine	ACO-Miner
Exact	Yes	Yes (Eq. 2)	No
Sound	Yes	Yes	Yes (proved)
Complete	Yes	Yes	No (Eq. 4)
Search strategy	BFS level-wise	DFS projections	Heuristic pheromone
DB scans	k scans	1 scan	1 scan + iterations

1.3 Research Objectives

1. Design an OOP system for mining maximal frequent itemsets from uncertain databases supporting multiple algorithms.
2. Support accumulated dynamic databases with batch appending and re-mining.
3. Implement and compare U-Apriori, UH-Mine, and ACO-Miner.
4. Evaluate on retail datasets of varying sizes.

CHAPTER 2. THEORETICAL BASIS

2.1 Problem Definition

The accumulated dynamic model extends Definition 2: at each time step a new batch B is appended to D without deleting historical data, and the system re-mines the full accumulated database.

2.2 System Overview

The proposed system is implemented in Java across four packages. Figure 3 shows the package structure.

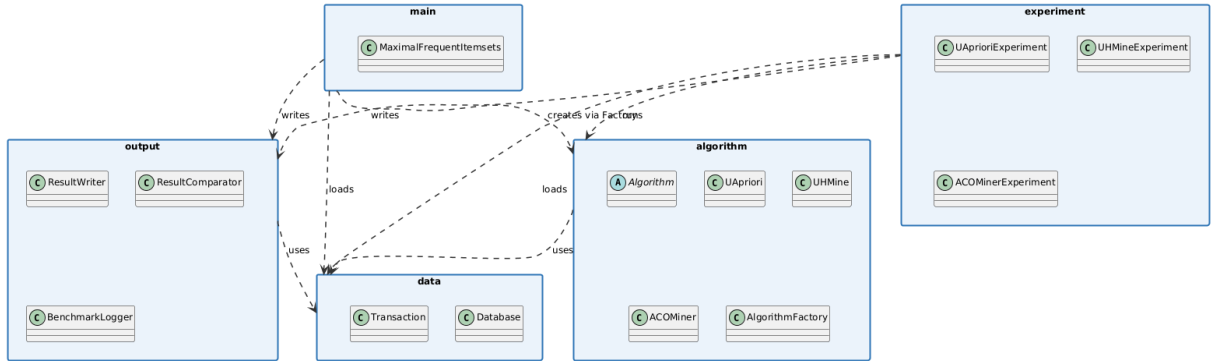


Figure 1: Package diagram of the proposed system

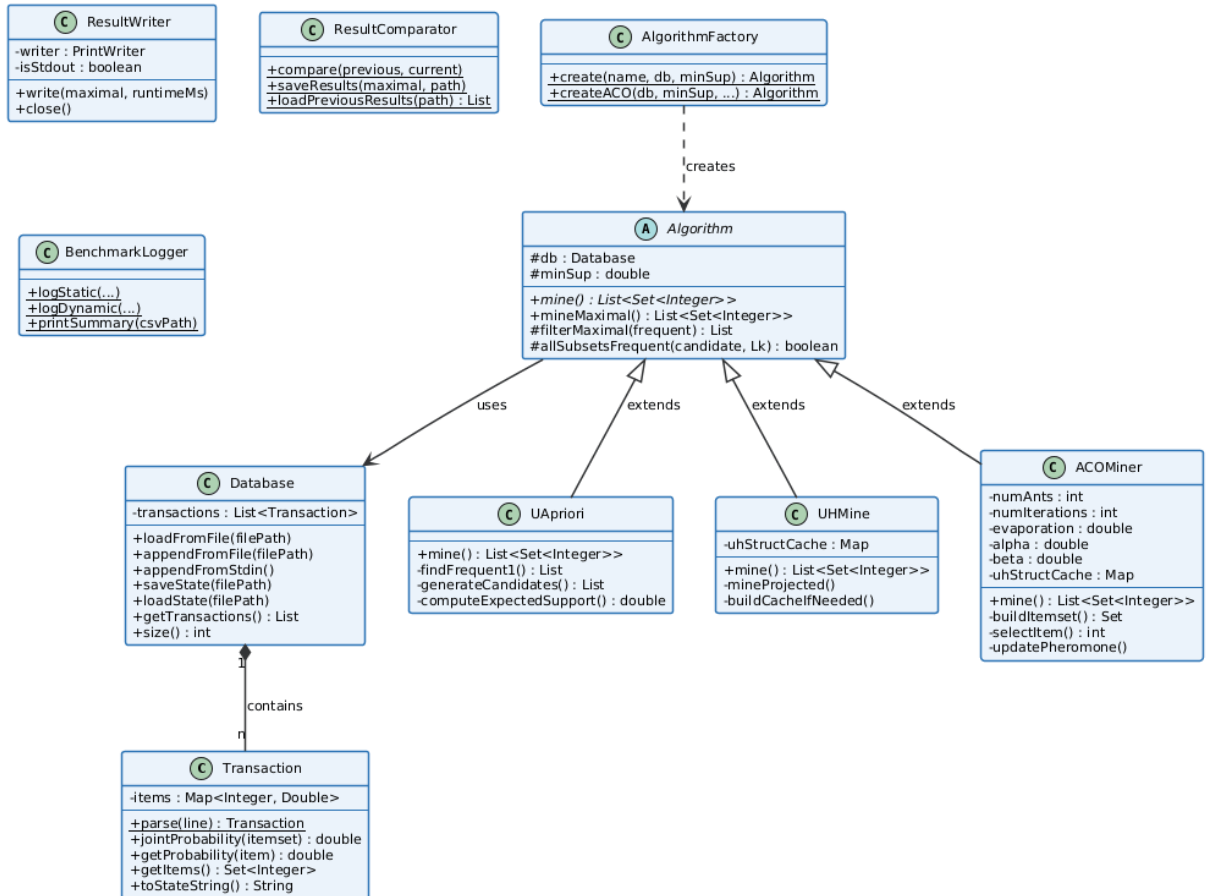


Figure 2: Package diagram of the proposed system

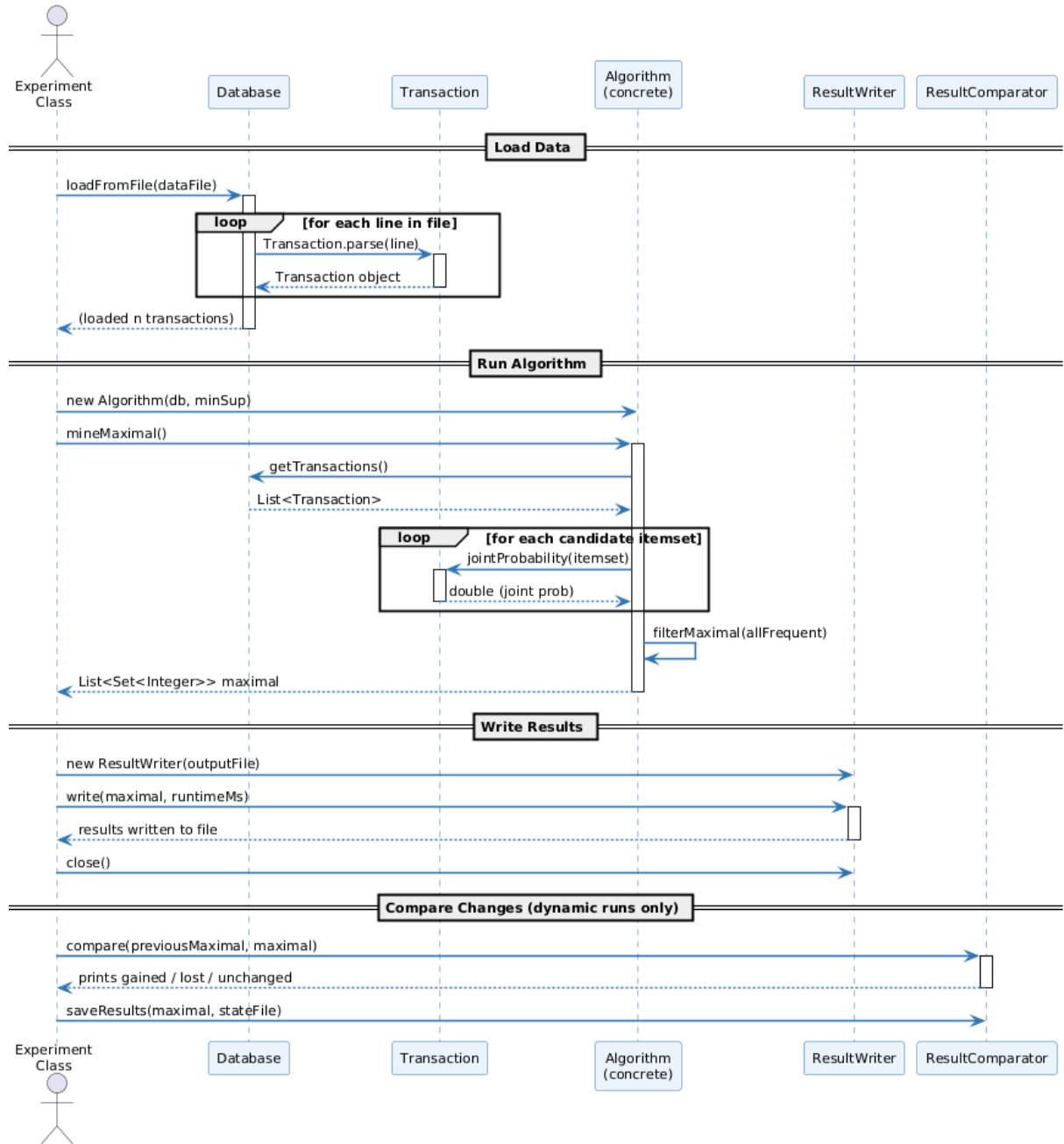


Figure 3: Package diagram of the proposed system

2.3 Data Package

2.3.1 Transaction Class

```

1 public class Transaction {
2     private final Map<Integer, Double> items; // item ID -> probability
3
4     // Parse "40:0.9 49:0.8 42:0.7" into a Transaction object
5     public static Transaction parse(String line) {
6         Map<Integer, Double> map = new HashMap<>();
7         for (String token : line.trim().split("\\s+")) {

```

```

8         String[] pair = token.split(":");
9         int item = Integer.parseInt(pair[0].trim());
10        double prob = Double.parseDouble(pair[1].trim());
11        map.put(item, prob);
12    }
13    return new Transaction(map);
14}

15
16// Core computation used by ALL algorithms: PROD p(x,t) for x in X
17// Returns 0.0 immediately if any item is absent (anti-monotone)
18public double jointProbability(Set<Integer> itemset) {
19    double prob = 1.0;
20    for (int item : itemset) {
21        if (!items.containsKey(item)) return 0.0;
22        prob *= items.get(item);
23    }
24    return prob; // Example: {40,49} in {40:0.9,49:0.8} => 0.72
25}

26
27// Serialize to "40:0.9 49:0.8" for Database.saveState()
28public String toStateString() {
29    StringBuilder sb = new StringBuilder();
30    for (Map.Entry<Integer, Double> e : items.entrySet()) {
31        if (sb.length() > 0) sb.append(" ");
32        sb.append(e.getKey()).append(":").append(e.getValue());
33    }
34    return sb.toString();
35}
36}

```

Listing 1: Transaction.java — key methods

2.3.2 Database Class

```

1 public class Database {
2     // Private final: only Database manages its own transactions
3     private final List<Transaction> transactions = new ArrayList<>();
4
5     // Load initial batch (Time 1)
6     public void loadFromFile(String filePath) throws IOException {
7         transactions.clear();
8         appendFromFile(filePath);
9     }
10
11    // Core accumulation: new data is NEVER deleted (dynamic model)
12    // Called at Time 2, 3, ... when new batches arrive
13    public void appendFromFile(String filePath) throws IOException {
14        try (BufferedReader br =
15            new BufferedReader(new FileReader(filePath))) {

```

```

16         String line;
17         while ((line = br.readLine()) != null) {
18             line = line.trim();
19             if (!line.isEmpty())
20                 transactions.add(Transaction.parse(line));
21         }
22     }
23 }
24
25 // Save full accumulated DB for next program run
26 public void saveState(String filePath) throws IOException {
27     try (BufferedWriter bw =
28         new BufferedWriter(new FileWriter(filePath))) {
29         for (Transaction t : transactions) {
30             bw.write(t.toStateString());
31             bw.newLine();
32         }
33     }
34 }
35
36 // Restore saved state (state file = same format as input)
37 public void loadState(String filePath) throws IOException {
38     transactions.clear();
39     appendFromFile(filePath);
40 }
41
42 // Unmodifiable view: algorithms cannot modify the list
43 public List<Transaction> getTransactions() {
44     return Collections.unmodifiableList(transactions);
45 }
46 }

```

Listing 2: Database.java — accumulation and state persistence

2.4 Algorithm Package

2.4.1 Algorithm Abstract Base Class

```

1 public abstract class Algorithm {
2     // Protected final: accessible to subclasses, set once
3     protected final Database db;
4     protected final double minSup;
5
6     // Every subclass implements its own mining strategy
7     public abstract List<Set<Integer>> mine();
8
9     // Default: mine() then filter for maximal
10    public List<Set<Integer>> mineMaximal() {

```

```

11     return filterMaximal(mine());
12 }
13
14 // Shared: remove non-maximal itemsets (all subclasses inherit)
15 // A is maximal if no frequent B exists with A proper subset of B
16 protected List<Set<Integer>> filterMaximal(
17     List<Set<Integer>> frequent) {
18     List<Set<Integer>> maximal = new ArrayList<>();
19     for (int i = 0; i < frequent.size(); i++) {
20         Set<Integer> A = frequent.get(i);
21         boolean isSubset = false;
22         for (int j = 0; j < frequent.size(); j++) {
23             if (i == j) continue;
24             Set<Integer> B = frequent.get(j);
25             if (B.size() > A.size() && B.containsAll(A)) {
26                 isSubset = true; break;
27             }
28         }
29         if (!isSubset) maximal.add(A);
30     }
31     return maximal;
32 }
33
34 // Shared: Apriori pruning (Theorem 1 Corollary)
35 // Returns false if any (k-1)-subset of candidate is not in Lk_1
36 protected boolean allSubsetsFrequent(
37     Set<Integer> candidate, List<Set<Integer>> Lk_1) {
38     List<Integer> items = new ArrayList<>(candidate);
39     for (int i = 0; i < items.size(); i++) {
40         Set<Integer> subset = new HashSet<>(candidate);
41         subset.remove(items.get(i));
42         if (!Lk_1.contains(subset)) return false;
43     }
44     return true;
45 }
46 }

```

Listing 3: Algorithm.java — abstract base with shared utilities

2.4.2 AlgorithmFactory Class

```

1 public class AlgorithmFactory {
2     // ONLY place that changes when a new algorithm is added
3     public static Algorithm create(String name, Database db,
4                                     double minSup) {
5         switch (name.toLowerCase()) {
6             case "uapriori": return new UApriori(db, minSup);
7             case "uhmine":   return new UHMine(db, minSup);
8             case "aco":      return new ACOMiner(db, minSup);

```

```

9         // case "ufpgrowth": return new UFPGrowth(db, minSup);
10        default: return null;
11    }
12 }
13 // Separate factory for ACO with custom parameters
14 public static Algorithm createACO(Database db, double minSup,
15     int ants, int iterations, double evaporation,
16     double alpha, double beta) {
17     return new ACOMiner(db, minSup,
18         ants, iterations, evaporation, alpha, beta);
19 }
20 }

```

Listing 4: AlgorithmFactory.java — Open-Closed Principle

2.4.3 U-Apriori

```

1  @Override
2  public List<Set<Integer>> mine() {
3      List<Transaction> transactions = db.getTransactions();
4      int n = transactions.size();
5      List<Set<Integer>> allFrequent = new ArrayList<>();
6
7      // Level 1: ExpSup({x}) = SUM p(x,t) / n
8      List<Set<Integer>> Lk = findFrequent1(transactions, n);
9      allFrequent.addAll(Lk);
10
11     int k = 2;
12     while (!Lk.isEmpty()) {
13         List<Set<Integer>> Ck = generateCandidates(Lk, k);
14         List<Set<Integer>> nextLk = new ArrayList<>();
15         for (Set<Integer> candidate : Ck) {
16             // Apriori pruning: skip if any (k-1)-subset not in Lk
17             if (!allSubsetsFrequent(candidate, Lk)) continue;
18             // Compute ExpSup = SUM PROD p(x,t) / n (Equation 1)
19             double support = computeExpectedSupport(
20                 transactions, candidate, n);
21             if (support >= minSup) {
22                 nextLk.add(candidate);
23                 allFrequent.add(candidate);
24             }
25         }
26         Lk = nextLk;
27         k++;
28     }
29     return allFrequent; // filterMaximal() called by mineMaximal()
30 }

```

Listing 5: UApriori.java — mine() BFS level-by-level

2.4.4 UH-Mine

```
1 private void mineProjected(  
2     List<Integer>      prefix,      // current itemset prefix  
3     Map<Integer, Double> prefixTxns, // projected txn list  
4     List<Integer>      candidates,  
5     int n, List<Set<Integer>> allFrequent) {  
6  
7     for (int i = 0; i < candidates.size(); i++) {  
8         int extItem = candidates.get(i);  
9         Map<Integer, Double> extItemTxns =  
10             uhStructCache.getDefault(extItem, Collections.emptyMap());  
11  
12         Map<Integer, Double> newTxns = new LinkedHashMap<>();  
13         double expSupSum = 0.0;  
14         for (Map.Entry<Integer, Double> e : prefixTxns.entrySet()) {  
15             int txnIdx = e.getKey();  
16             double prefProb = e.getValue(); // joint prob of prefix  
17             Double extProb = extItemTxns.get(txnIdx);  
18             if (extProb != null) {  
19                 // Intersection: multiply joint prob  
20                 double jointProb = prefProb * extProb;  
21                 newTxns.put(txnIdx, jointProb);  
22                 expSupSum += jointProb;  
23             }  
24         }  
25         double expSup = expSupSum / n; // identical to U-Apriori  
26  
27         if (expSup >= minSup) {  
28             List<Integer> newPrefix = new ArrayList<>(prefix);  
29             newPrefix.add(extItem);  
30             allFrequent.add(new HashSet<>(newPrefix));  
31             List<Integer> remaining =  
32                 candidates.subList(i + 1, candidates.size());  
33             if (!remaining.isEmpty() && !newTxns.isEmpty())  
34                 mineProjected(newPrefix, newTxns,  
35                             remaining, n, allFrequent);  
36         }  
37         // expSup < minSup => anti-monotone => DFS terminates branch  
38     }  
39 }
```

Listing 6: UHMine.java — mineProjected() DFS core

2.4.5 ACO-Miner

```
1 // One ant builds one itemset (SOUND: every item checked ExpSup >= minSup)  
2 private Set<Integer> buildItemset(List<Integer> freqItems,
```

```

3      Map<Integer, Double> pheromone,
4      Map<Integer, Double> heuristic, int n, Random random) {
5      Set<Integer>          itemset    = new HashSet<>();
6      List<Integer>         candidates = new ArrayList<>(freqItems);
7      Map<Integer, Double> projTxns  = null;
8      while (!candidates.isEmpty()) {
9          int selected = selectItem(candidates, pheromone,
10                                   heuristic, random);
11          // Equation (4):  $P(i) = \tau^{\alpha} * \eta^{\beta} / \text{SUM}$ 
12          Map<Integer, Double> newProjTxns =
13              projectTransactions(projTxns, selected);
14          double expSup = sumSupport(newProjTxns) / n; // Equation (1)
15          if (expSup >= minSup) {
16              itemset.add(selected);
17              projTxns = newProjTxns;
18          }
19          candidates.remove((Integer) selected);
20      }
21      return itemset; // SOUNDNESS: every item passed ExpSup >= minSup
22 }
23
24 // Roulette wheel selection proportional to  $\tau^{\alpha} * \eta^{\beta}$ 
25 private int selectItem(List<Integer> candidates,
26                        Map<Integer, Double> pheromone,
27                        Map<Integer, Double> heuristic, Random random) {
28     double[] weights = new double[candidates.size()];
29     double total = 0.0;
30     for (int i = 0; i < candidates.size(); i++) {
31         int item = candidates.get(i);
32         double ph = Math.pow(
33             pheromone.getDefault(item, INITIAL_PHEROMONE), alpha);
34         double he = Math.pow(
35             heuristic.getDefault(item, 0.0), beta);
36         weights[i] = ph * he;
37         total += weights[i];
38     }
39     double spin = random.nextDouble() * total;
40     double cum = 0.0;
41     for (int i = 0; i < candidates.size(); i++) {
42         cum += weights[i];
43         if (cum >= spin) return candidates.get(i);
44     }
45     return candidates.get(candidates.size() - 1);
46 }
47
48 // Pheromone update: evaporate then reinforce
49 private void updatePheromone(List<Integer> freqItems,
50                              Map<Integer, Double> pheromone,
51                              List<Set<Integer>> iterFound, int n) {

```

```

52     for (int item : freqItems)
53         pheromone.put(item, (1.0 - evaporation) * pheromone.get(item));
54     for (Set<Integer> itemset : iterFound) {
55         double reward = computeExpSup(itemset, n);
56         for (int item : itemset)
57             pheromone.merge(item, reward, Double::sum);
58     }
59 }

```

Listing 7: ACOMiner.java — buildItemset() and selectItem()

2.5 Output Package

2.5.1 ResultWriter

```

1 public class ResultWriter {
2     private final PrintWriter writer;
3     private final boolean      isStdout; // never close System.out
4
5     public ResultWriter(String filePath) throws IOException {
6         if (filePath != null) {
7             writer = new PrintWriter(
8                 new BufferedWriter(new FileWriter(filePath)));
9             isStdout = false;
10        } else {
11            // Wrap System.out but DO NOT close it afterward
12            // Closing System.out would silence ResultComparator output
13            writer = new PrintWriter(
14                new BufferedWriter(new OutputStreamWriter(System.out)));
15            isStdout = true;
16        }
17    }
18    public void close() {
19        writer.flush();
20        if (!isStdout) writer.close(); // only close file writers
21    }
22 }

```

Listing 8: ResultWriter.java — isStdout guard

2.5.2 ResultComparator

```

1 public static void compare(List<Set<Integer>> previous,
2                           List<Set<Integer>> current) {
3     List<Set<Integer>> gained = new ArrayList<>();
4     List<Set<Integer>> lost   = new ArrayList<>();
5     List<Set<Integer>> unch   = new ArrayList<>();

```

```

6
7     for (Set<Integer> s : current)
8         (containsSet(previous, s) ? unch : gained).add(s);
9     for (Set<Integer> s : previous)
10        if (!containsSet(current, s)) lost.add(s);
11    // Lost: no longer maximal or no longer frequent after new batch
12
13    System.out.println("Unchanged : " + unch.size());
14    for (Set<Integer> s : unch)      System.out.println(" = " + s);
15    System.out.println("Gained      : " + gained.size());
16    for (Set<Integer> s : gained)    System.out.println(" + " + s);
17    System.out.println("Lost        : " + lost.size());
18    for (Set<Integer> s : lost)      System.out.println(" - " + s);
19 }

```

Listing 9: ResultComparator.java — change tracking

2.5.3 BenchmarkLogger

```

1  // Static run: writes to benchmark.csv only
2  public static void logStatic(String csvPath, String algorithm,
3      String dataFile, int transactions, double minSup,
4      int maximalCount, long runtimeMs) throws IOException {
5      writeRow(csvPath, "static", new File(dataFile).getName(),
6          algorithm, 0, 0, transactions, "-",
7          minSup, maximalCount, runtimeMs);
8  }
9
10 // Dynamic run: writes to benchmark.csv (runtime)
11 //      AND to benchmark_sets.csv (gained/lost itemset details)
12 public static void logDynamic(String csvPath, String algorithm,
13     String baseFile, List<String> addedBatchFiles,
14     int txnBefore, int txnAfter, double minSup,
15     List<Set<Integer>> previousMaximal,
16     List<Set<Integer>> currentMaximal,
17     long runtimeMs) throws IOException {
18     // Main CSV: runtime summary
19     writeRow(csvPath, "dynamic", ...);
20     // Sets CSV: itemset change details (gained/lost/unchanged)
21     String setsPath = setsFilePath(csvPath);
22     writeSetsRow(setsPath, algorithm, txnBefore, txnAfter,
23         batchSummary, previousMaximal, currentMaximal);
24 }

```

Listing 10: BenchmarkLogger.java — dual CSV logging

2.6 Accumulated Dynamic Database Model

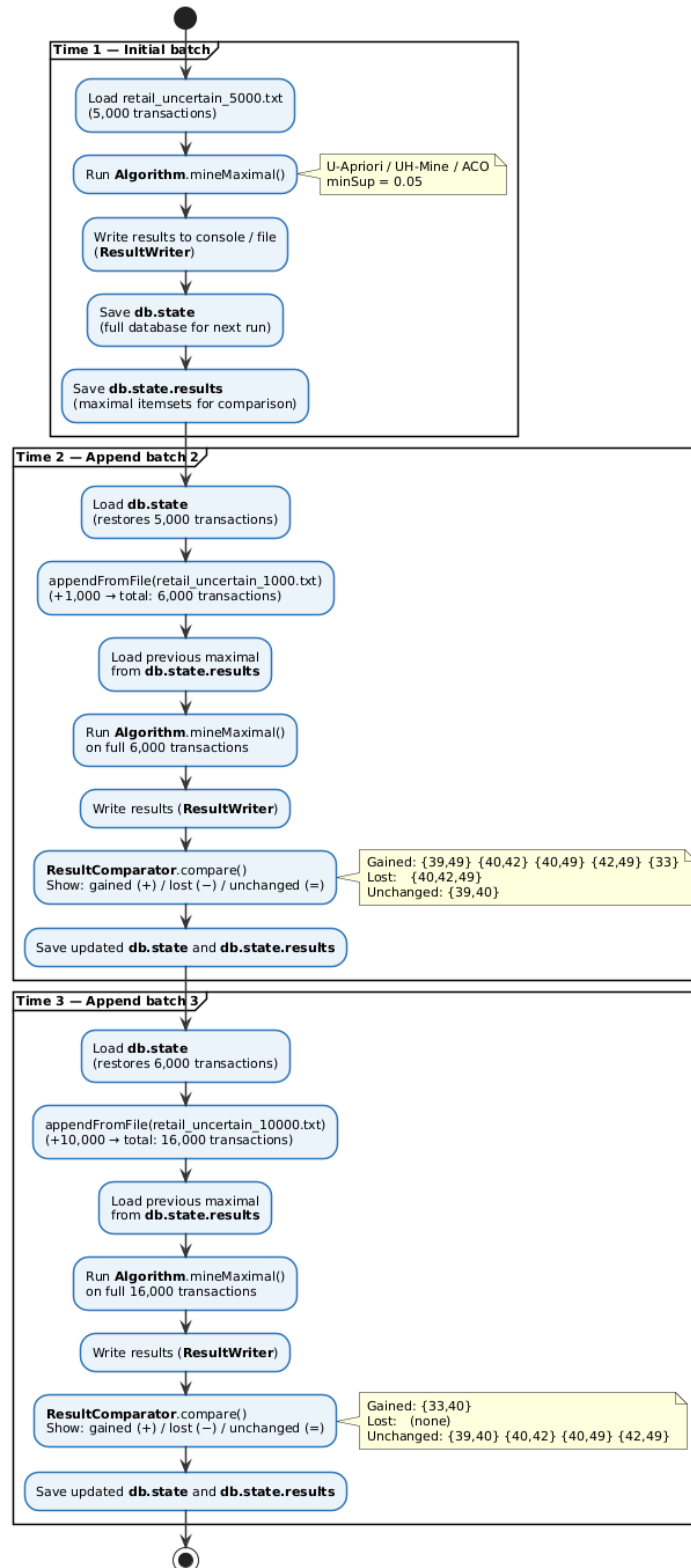


Figure 4: Sequence diagram of the mining process

2.7 Class Summary

Table 3: Class responsibilities

Class	Package	Responsibility
Transaction	data	Owns <code>jointProbability()</code>
Database	data	Loads, accumulates, saves/loads state
Algorithm	algorithm	Abstract base; <code>filterMaximal()</code>
UApriori	algorithm	Exact BFS + Apriori pruning
UHMine	algorithm	Exact DFS + projected list intersection
ACOMiner	algorithm	Approximate ant colony search
AlgorithmFactory	algorithm	Factory — single registration point
ResultWriter	output	Writes results to console or file
ResultComparator	output	Tracks gained/lost itemsets
BenchmarkLogger	output	Logs runtime and details to CSV
MaximalFrequentItemsets	main	CLI entry point
UAprioriExperiment	experiment	Static + dynamic; writes 4 files
UHMineExperiment	experiment	Static + dynamic; writes 4 files
ACOMinerExperiment	experiment	Static + dynamic + coverage

CHAPTER 3. PROPOSED MODEL

3.1 Hardware and Software Environment

- Operating System: Windows 10/11
- Java Version: Java SE 17 or later
- Implementation: Java, 4 packages (data, algorithm, output, main)

3.2 Datasets

Table 4: Experimental datasets

Dataset	Transactions	Description
retail_uncertain_1000.txt	1,000	Small — baseline
retail_uncertain_5000.txt	5,000	Medium
retail_uncertain_10000.txt	10,000	Large
retail_uncertain.txt	88,162	Full dataset

3.3 Evaluation Metrics

- **Runtime (ms):** wall-clock time, database load to output.
- **Maximal itemset count:** total discovered.
- **Correctness:** match U-Apriori (ground truth).
- **Coverage (ACO only):** % of exact maximal itemsets found.

3.4 Dynamic Accumulation Experiment

1. **Time 1:** Mine retail_uncertain_5000.txt (5,000 transactions). Save state.
2. **Time 2:** Append retail_uncertain_1000.txt. Total: 6,000. Re-mine.
3. **Time 3:** Append retail_uncertain_10000.txt. Total: 16,000. Re-mine.

```
1  # Compile all packages
2  javac main\*.java data\*.java algorithm\*.java output\*.java
3
4  # Time 1 -- initial mine
5  java main.MaximalFrequentItemsets uapriori \
6      ..\data\retail_uncertain_5000.txt \
7      --minsup 0.05 --save-state db.state --benchmark benchmark.csv
8
9  # Time 2 -- load state, append, re-mine
10 java main.MaximalFrequentItemsets uapriori \
11     --load-state db.state \
12     --add ..\data\retail_uncertain_1000.txt \
13     --minsup 0.05 --save-state db.state --benchmark benchmark.csv
14
15 # All three algorithms on 5000-transaction dataset
16 java main.MaximalFrequentItemsets uapriori \
17     ..\data\retail_uncertain_5000.txt --minsup 0.05 --benchmark benchmark.csv
18 java main.MaximalFrequentItemsets uhmine \
19     ..\data\retail_uncertain_5000.txt --minsup 0.05 --benchmark benchmark.csv
20 java main.MaximalFrequentItemsets aco \
21     ..\data\retail_uncertain_5000.txt --minsup 0.05 --benchmark benchmark.csv
22
23 # Print benchmark summary
24 java main.MaximalFrequentItemsets --benchmark-summary benchmark.csv
```

Listing 11: Benchmark commands used in experiments

CHAPTER 4. EXPERIMENTS

4.1 Algorithm Correctness Verification

Table 5: Correctness verification ($\theta = 0.05$)

Dataset	U-Apriori	UH-Mine	ACO	UH-Mine = U-Apriori?
1,000 transactions	6	6	5	Yes
5,000 transactions	4	4	2	Yes
10,000 transactions	6	6	5	Yes
88,162 transactions	7	7	4	Yes

Finding 1 — UH-Mine correctness confirmed

UH-Mine produced identical results to U-Apriori on every dataset. ACO-Miner found fewer itemsets — consistent with its approximate nature.

4.2 Runtime Performance

Table 6: Runtime comparison ($\theta = 0.05$, ms)

Dataset	U-Apriori	UH-Mine	ACO-Miner
1,000	32	45	150
5,000	101	211	577
10,000	103	151	1,056
88,162	369	856	12,435

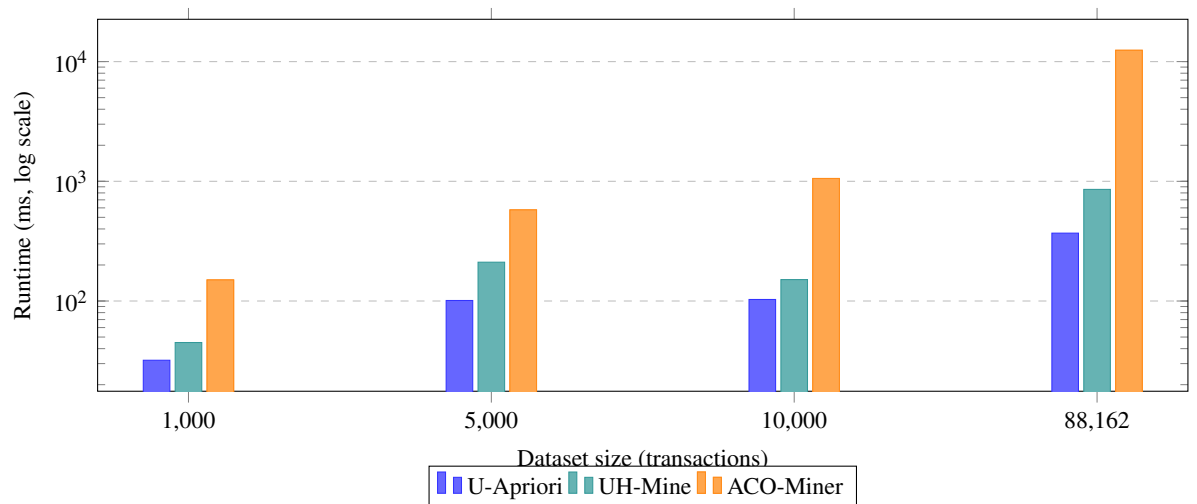


Figure 5: Runtime comparison (log scale, $\theta = 0.05$)

U-Apriori is faster than UH-Mine at $\theta = 0.05$ because this dataset produces few frequent itemsets (4–7 maximal sets), so candidate generation terminates quickly. UH-Mine’s overhead is not justified for such compact result sets. ACO-Miner is consistently slowest ($33.6\times$ U-Apriori on 88,162 transactions) because it always completes all $A \times I$ iterations.

4.3 Effect of Minimum Support Threshold

Table 7: Effect of θ on full dataset (88,162 transactions)

θ	U-Apriori		UH-Mine		ACO-Miner	
	Sets	ms	Sets	ms	Sets	ms
0.01	49	9,165	49	1,115	9	16,376
0.05	7	358	7	784	4	12,435
0.10	4	301	4	784	1	12,691
0.20	2	173	2	725	–	–

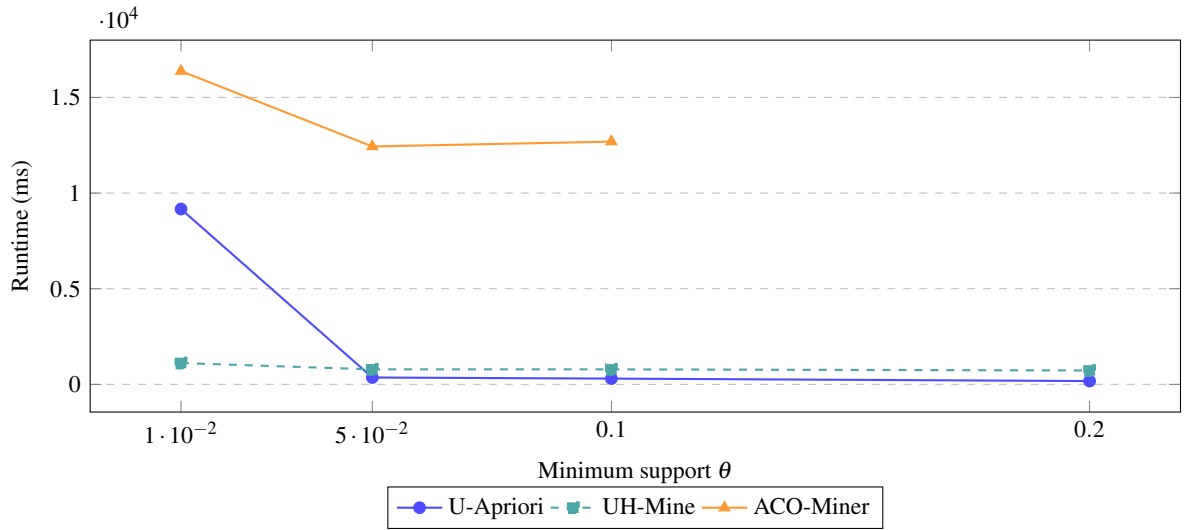


Figure 6: Runtime vs. θ (88,162 transactions)

Finding 2 — UH-Mine advantage at lower thresholds

At $\theta = 0.01$, UH-Mine is $8.2\times$ faster than U-Apriori (1,115 ms vs 9,165 ms), both finding 49 identical maximal itemsets.

4.4 Dynamic Accumulation Results

Table 8: Dynamic accumulation results ($\theta = 0.05$)

Step	Algorithm	Transactions	Sets	Gained	Lost
Time 1	U-Apriori	5,000	4	—	—
Time 1	UH-Mine	5,000	4	—	—
Time 1	ACO-Miner	5,000	2	—	—
Time 2	U-Apriori	6,000	6	5	1
Time 2	UH-Mine	6,000	6	5	1
Time 2	ACO-Miner	6,000	4	3	1
Time 3	U-Apriori	16,000	5	1	0
Time 3	UH-Mine	16,000	5	1	0
Time 3	ACO-Miner	16,000	5	1	0

Table 9: Itemset changes tracked by ResultComparator ($\theta = 0.05$)

Step	Algorithm	Gained (+)	Lost (−)	Unchanged (=)
Time 2 5000→6000	U-Apriori	{39,49} {40,49} {42,49} {33}	{40,42} {40,42,49}	{39,40}
	UH-Mine	{40,49} {42,49} {39,49} {33}	{40,42} {40,42,49}	{39,40}
	ACO-Miner	{40,42} {42,49}	{40,49} {40,42,49}	{39,40}
Time 3 6000→16000	U-Apriori	{33,40}	(none)	{39,40} {40,42} {40,49} {42,49}
	UH-Mine	{33,40}	(none)	{40,49} {40,42} {39,40} {42,49}
	ACO-Miner	{33,40}	(none)	{40,42} {40,49} {42,49} {39,40}

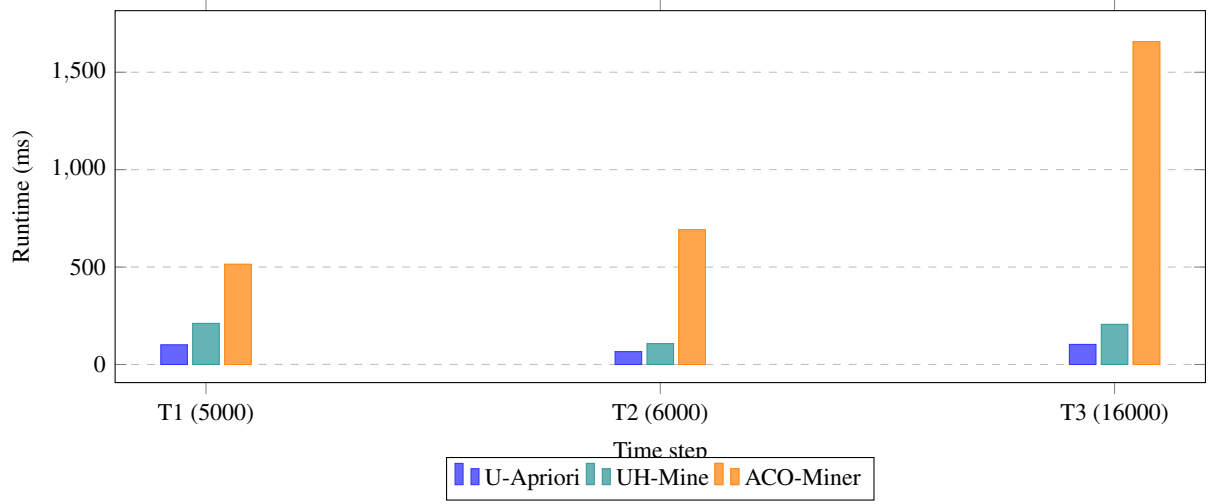


Figure 7: Runtime across dynamic accumulation steps ($\theta = 0.05$)

Finding 3 — dynamic results are non-monotone

At Time 2, $\{40, 42, 49\}$ was lost and replaced by 5 two-itemsets. At Time 3, only $\{33, 40\}$ was gained with no losses. ResultComparator correctly tracked all changes.

4.5 ACO-Miner Coverage Analysis

Table 10: ACO coverage vs. runtime (retail_uncertain_5000.txt, $\theta = 0.05$, exact = 4 itemsets)

Ants	Iter.	$A \times I$	Found	Coverage	ms
5	10	50	1	25%	159
20	50	1,000	2	50%	514
50	100	5,000	3	75%	1,880
100	200	20,000	4	100%	7,495

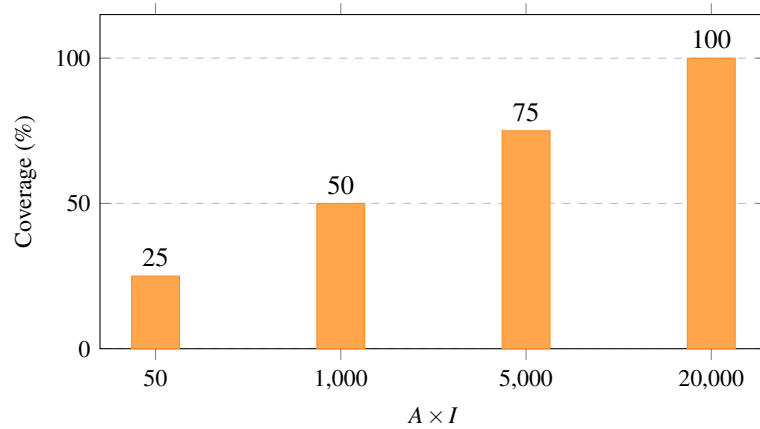


Figure 8: ACO-Miner coverage vs. $A \times I$ ($\theta = 0.05$)

Finding 4 — ACO coverage-speed trade-off

100% coverage at $A \times I = 20,000$ costs $74\times$ U-Apriori runtime. Default ($A \times I = 1,000$): 50% coverage, $5\times$ slower.

4.6 Discussion

U-Apriori vs. UH-Mine. Both exact algorithms produced identical results on all datasets. U-Apriori is faster at $\theta = 0.05$ due to the compact result set. UH-Mine is $8.2\times$ faster at $\theta = 0.01$, confirming its advantage when the search space is large.

ACO-Miner. Consistently slowest; runtime does not decrease with higher θ because it always completes all iterations. At $\theta = 0.01$ on 88,162 transactions, ACO found only 9 of 49 exact itemsets (18%).

Dynamic accumulation. Result sets can shrink as data accumulates, not just grow. The fragmentation of $\{40, 42, 49\}$ at Time 2 shows sensitivity of longer itemsets to the statistical composition of new batches.

CHAPTER 5. CONCLUSION

5.1 Conclusion

This project presented an object-oriented system for mining maximal frequent itemsets from accumulated dynamic uncertain databases. Three algorithms were designed, implemented, and evaluated. The system follows OOP design principles including encapsulation, the Single Responsibility Principle, and the Open-Closed Principle.

Key findings: (1) UH-Mine is exact and identical to U-Apriori, confirmed on all datasets. (2) U-Apriori is faster at $\theta = 0.05$ on compact datasets; UH-Mine is $8.2\times$ faster at $\theta = 0.01$ when the search space is large. (3) Dynamic result sets are non-monotone — the 3-itemset $\{40, 42, 49\}$ fragmented at Time 2. (4) ACO-Miner achieves 100% coverage at $A \times I = 20,000$ but at $74\times$ the U-Apriori runtime.

5.2 Contributions

1. Unified OOP framework supporting multiple algorithms through a common abstract base class and factory pattern.
2. Separate experiment classes (one per algorithm) for reproducibility, each writing results and parameters to files.
3. Verified UH-Mine correctness against U-Apriori across all datasets and thresholds.
4. ACO-Miner implementation with coverage measurement against the exact ground truth.
5. Accumulated dynamic model with state persistence and change tracking via `ResultComparator`.

5.3 Limitations

Re-mining scans the full accumulated database, which is expensive at low thresholds. ACO-Miner uses a fixed random seed. Datasets use simulated probabilities rather than real uncertain data. Memory measurement via `Runtime` is approximate due to JVM GC.

5.4 Future Development

- Incremental update strategies to avoid full re-mining.
- Additional algorithms via the existing `AlgorithmFactory`.
- Evaluation on real uncertain datasets (medical, RFID).
- Sliding window model for expiring old transactions.
- ACO parameter optimization and parallel implementation.

REFERENCES

- [1] Agrawal, R., & Srikant, R. (1993). Mining association rules between sets of items in large databases. *Proc. ACM SIGMOD*, 207–216.
- [2] Agrawal, R., & Srikant, R. (1994). Fast algorithms for mining association rules. *Proc. VLDB*, 487–499.
- [3] Aggarwal, C. C., Li, Y., Wang, J., & Wang, J. (2009). Frequent pattern mining with uncertain data. *Proc. KDD*, 29–38.
- [4] Aggarwal, C. C., & Yu, P. S. (2009). A survey of uncertain data algorithms and applications. *IEEE TKDE*, 21(5), 609–623.
- [5] Chui, C.-K., Kao, B., & Hung, E. (2007). Mining frequent itemsets from uncertain data. *Proc. PAKDD*, 47–58.
- [6] Han, J., Pei, J., & Yin, Y. (2000). Mining frequent patterns without candidate generation. *Proc. ACM SIGMOD*, 1–12.
- [7] Leung, C. K. S., Carmichael, C. L., & Hao, B. (2007). Efficient mining of frequent patterns from uncertain data. *Proc. ICDMW*, 489–494.
- [8] Malipatil, S., & Hanumantha Reddy, T. (2023). Discovery of interesting frequent item sets in an uncertain database using ant colony optimization. *IJCA*, 45(11), 673–679.